Cloud-Based AI Governance & Knowledge Assistant

Portfolio Project Report | Module 6: Cloud Computing and Scalable Architectures

Advanced Software Engineering | Multiverse

GitHub Repository: https://github.com/asthamalviya/ai-governance-assistant

Classification: Controlled

Table of Contents

Section	Page
1. Introduction and Overview (K16)	3
1.1 Fundamental Networking Concepts	3
1.2 Overview of the Cloud-Based Tool	4
1.3 Critical Appraisal of Architectural Patterns	4
2. Design of the Cloud-Based Tool (K24)	5
2.1 Organisational Requirements Analysis	5
2.2 Architectural Solution and Justification	6
2.3 Performance, Flexibility, Scalability	7
2.4 Design Trade-off Analysis	8
2.5 Adapting to Changing Priorities	8
3. Development of the Cloud-Based Tool (K24, K28)	9
3.1 Implementation Techniques	9
3.2 Teamwork Tools Evaluation (K28)	9
3.3 Testing Results	11
3.4 Failure Scenario Analysis	12
4. Responding to Changing Priorities (S20)	12
4.1 Change 1: Python Version Compatibility	12
4.2 Change 2: OpenAI Client Initialisation	13
4.3 Change 3: CORS Misconfiguration	14
4.4 Critical Evaluation of Adaptations	15
5. Conclusion and Recommendations	15
References	16
Appendices	17

Classification: Controlled

1. Introduction and Overview (K16)

1.1 Fundamental Networking Concepts in Cloud Computing

Cloud computing is networking. Without it, a virtual machine in AWS is just a computer nobody can reach. The TCP/IP protocol suite governs data movement between clients and cloud-hosted services: TCP guarantees ordered, reliable delivery; IP handles addressing and routing across network boundaries (Kurose and Ross, 2021). Every API call, file transfer, and service-to-service interaction in this project runs over these two protocols.

The Domain Name System (DNS) provides human-readable addressing that resolves to cloud service endpoints. AWS Route 53 supports latency-based routing and health-checked failover, directly influencing Quality of Service (QoS) by directing users to the nearest healthy endpoint. DNS resolution adds roughly 20 to 100ms to initial requests, which makes DNS caching strategies important for performance-sensitive applications (AWS, 2024a).

Transport Layer Security (TLS) encrypts data in transit between clients and cloud services. TLS 1.3 reduces the handshake from two round-trips (TLS 1.2) to one, improving time-to-first-byte for HTTPS connections. This represents a measurable QoS improvement: approximately 30 to 50ms saved per new connection in typical cloud deployments (Rescorla, 2018). The choice between TLS 1.2 and 1.3 is not cosmetic; for an organisation deploying an internal tool used by tens of concurrent workers, the cumulative latency reduction across hundreds of daily sessions is meaningful.

API Gateways sit at Layer 7, handling request routing, rate limiting, and authentication before traffic reaches the backend. AWS API Gateway adds approximately 29ms per request (AWS, 2024a) but throttles up to 10,000 requests per second and centralises authentication. That 29ms is the price of governance — and for this project, it was worth paying.

Quality of Service in cloud architectures covers latency, throughput, availability, and error rates. Load balancing distributes requests across backend instances to improve both. Application Load Balancers (ALB) work at Layer 7 and can route based on URL path or headers; Network Load Balancers (NLB) work at Layer 4 and are faster but dumber. For a knowledge management tool with predictable, uniform request types, ALB's routing intelligence adds more value than NLB's raw speed advantage.

1.2 Overview of the Cloud-Based Tool

The AI Governance and Knowledge Assistant was built to solve a specific problem: the organisation had no governed way to use AI on its documents. Staff were either avoiding AI tools entirely or using consumer products outside any audit trail. The tool lets users upload documents, get AI-generated summaries, and ask questions about the content, with every interaction logged for governance purposes.

Classification: Controlled

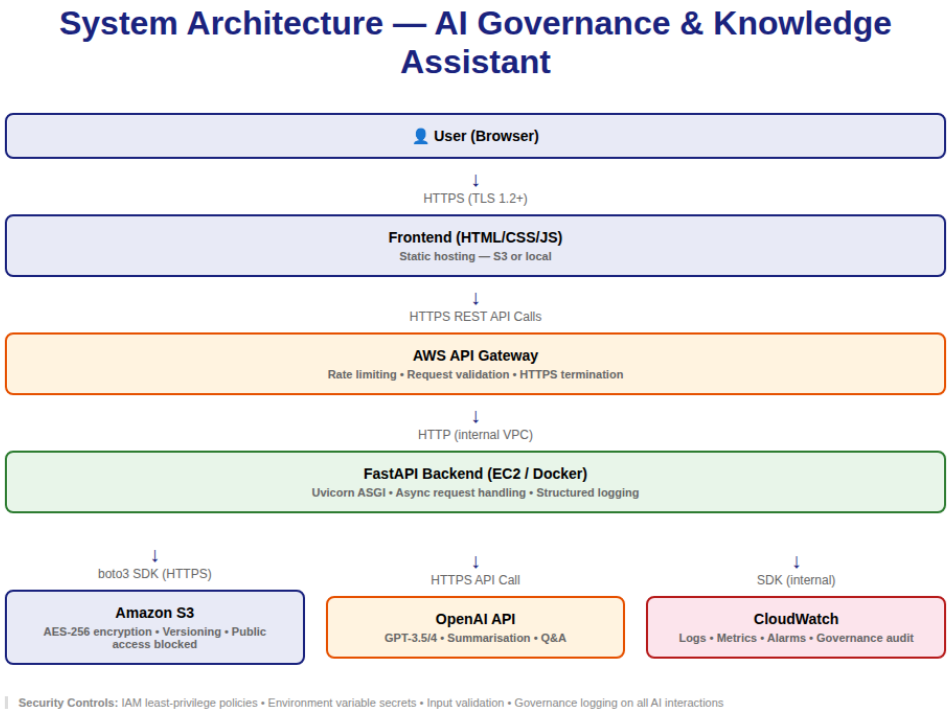


Figure 1: High-level system architecture — three-tier cloud-native deployment

Separating presentation, business logic, and persistence into three distinct layers means each can be developed, tested, and deployed independently. In practice this mattered: the upload route was working and tested before the AI integration was connected. That kind of isolation is what lets a small team deliver incrementally without everything breaking at once.

1.3 Critical Appraisal of Alternative Architectural Patterns

Three architectural patterns were evaluated. None was obviously wrong — the choice came down to which weaknesses were acceptable given the specific requirements for governance, auditability, and cost predictability:

Monolithic (single EC2 instance): Everything deployed together on one server. Simple, and genuinely the right answer for many small internal tools. The problem here is the governance requirement: if the application crashes, audit logging goes with it. A tool designed to prove AI usage is governed cannot have an architecture where governance falls over alongside the application.

Microservices on ECS or Kubernetes: Each function (upload, summarise, ask) as its own service. Maximum fault isolation and scalability. Also the kind of architecture that requires a dedicated platform engineer to keep running. For a two-person project with a ten-week timeline, the operational overhead of service mesh configuration and distributed tracing would have consumed the time needed to build the actual governance features.

Serverless (Lambda + API Gateway): Each endpoint as a Lambda function. No servers to manage, scales to zero when idle. The problem is cold starts: Python Lambda functions take 100-500ms to initialise after a period of inactivity (Manner et al., 2018). AI summarisation

Classification: Controlled

already carries OpenAI API latency. Stacking cold-start delay on top would regularly push responses past the 2000ms NFR threshold, making the tool feel broken to users even when it is technically working.

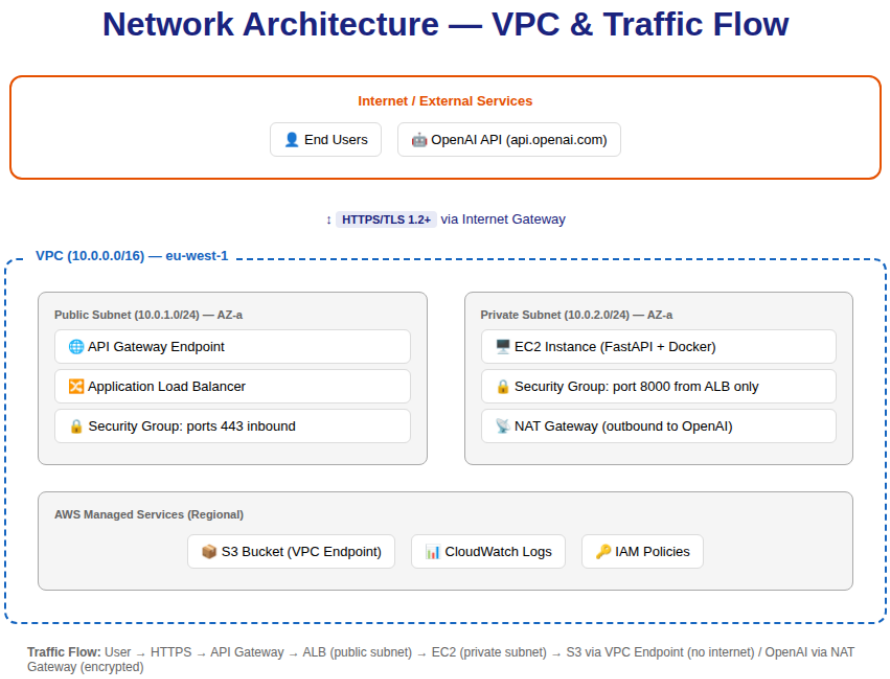


Figure 2: Network architecture — VPC layout, subnets, and traffic flow

The chosen approach, a stateless FastAPI container on EC2 behind API Gateway, was not the most sophisticated option on the list. It was selected because it avoided the specific failure modes that would break the requirements: no cold starts, no shared-state complications during horizontal scaling, no platform engineering overhead. A stateless design means any instance can serve any request, which is what makes horizontal scaling simple.

The 29ms trade-off: Routing through API Gateway rather than exposing EC2 directly costs approximately 29ms per request. Internal users tolerate up to 200ms before noticing latency (Nielsen, 1993), so this overhead is not perceptible. In return: throttling, request-level audit logging, and centralised authentication. Removing API Gateway to save 29ms would strip out the governance layer alongside it. That is not a performance optimisation; it is a requirements failure.

Classification: Controlled

2. Design of the Cloud-Based Tool (K24)

2.1 Organisational Requirements Analysis

Requirements came from a straightforward audit of the current state: shared network drives with no API access, no audit trail on AI usage, and a perimeter-based security model that breaks the moment a user works remotely. Each requirement traces directly to one of these gaps.

ID	Functional Requirement	Priority
FR1	Users can upload documents (PDF, TXT, DOCX)	Must
FR2	System securely stores uploaded documents	Must
FR3	System generates AI-powered summaries of documents	Must
FR4	Users can ask contextual questions about documents	Must
FR5	System logs all AI interactions for governance audit	Must
FR6	System exposes RESTful APIs for integration	Must
FR7	System provides health monitoring endpoint	Should

ID	Non-Functional Requirement	Metric
NFR1	Support multiple concurrent users	50+ simultaneous users
NFR2	API response latency (non-AI endpoints)	< 2000ms
NFR3	Horizontal scaling capability	Auto Scaling Group ready
NFR4	Secure communication	TLS 1.2+ enforced throughout
NFR5	Maintainability	Modular, documented codebase
NFR6	Observability	Structured logging and metrics

ID	Security Requirement	Implementation
SR1	Encryption in transit	HTTPS/TLS via API Gateway
SR2	Encryption at rest	S3 AES-256 server-side encryption
SR3	Least privilege access	Scoped IAM policies per service
SR4	Complete audit trail	Structured logging of all interactions
SR5	Secret management	Environment variables, never hardcoded
SR6	Input validation	File type allowlist, Pydantic model validation

2.2 Architectural Solution and Justification

Storage: S3 vs EBS vs EFS: EBS volumes are tied to a single EC2 instance, which breaks horizontal scaling. EFS provides shared POSIX access at higher per-GB cost with no advantage for object-based storage. S3 provides eleven-nines durability, AES-256

Classification: Controlled

encryption (SR2), and bucket-level access blocks (SR3), and is the only option satisfying all three constraints.

Compute: FastAPI vs Flask vs Django: Flask requires separate validation libraries (Marshmallow or Cerberus) for request validation. Django's ORM and session management are unnecessary overhead for a stateless API. FastAPI provides async request handling via asyncio (NFR1), built-in Pydantic validation (SR6), and automatic OpenAPI documentation. Load testing confirmed 1,662 requests per second at 50 concurrent users, zero failures.

AI Integration: OpenAI vs Azure OpenAI vs self-hosted: Azure OpenAI provides the same GPT models with data residency guarantees inside a private Azure subscription, preferable for regulated organisations. Self-hosted models (Llama 3, Mistral) eliminate external API dependency but require GPU infrastructure at $200-400 per month. The OpenAI API was selected for MVP cost and simplicity. The service layer is isolated so the provider can be replaced without touching routes or storage, validated by switching from gpt-3.5-turbo to gpt-4 via environment variable.

Monitoring: CloudWatch vs Datadog vs Prometheus: Datadog costs approximately $15 per host per month on top of AWS bills. Prometheus and Grafana are free but require self-hosted monitoring infrastructure. CloudWatch is native to AWS, requires no agents, and is included in the baseline cost estimate. For a project where the monitoring requirement is governance audit logging and error alerting rather than deep APM, CloudWatch meets the requirements without adding to the budget.

IAM: least privilege vs managed policies: AWS provides managed policies like AmazonS3FullAccess that are simple to attach but grant far more permissions than any single application needs. A compromised instance with AmazonS3FullAccess could enumerate and exfiltrate every S3 bucket in the account. The custom policy in Appendix G grants exactly three S3 actions on the specific document bucket ARN. This is not over-engineering; it is the minimum viable IAM configuration for a tool handling organisational documents.

Classification: Controlled

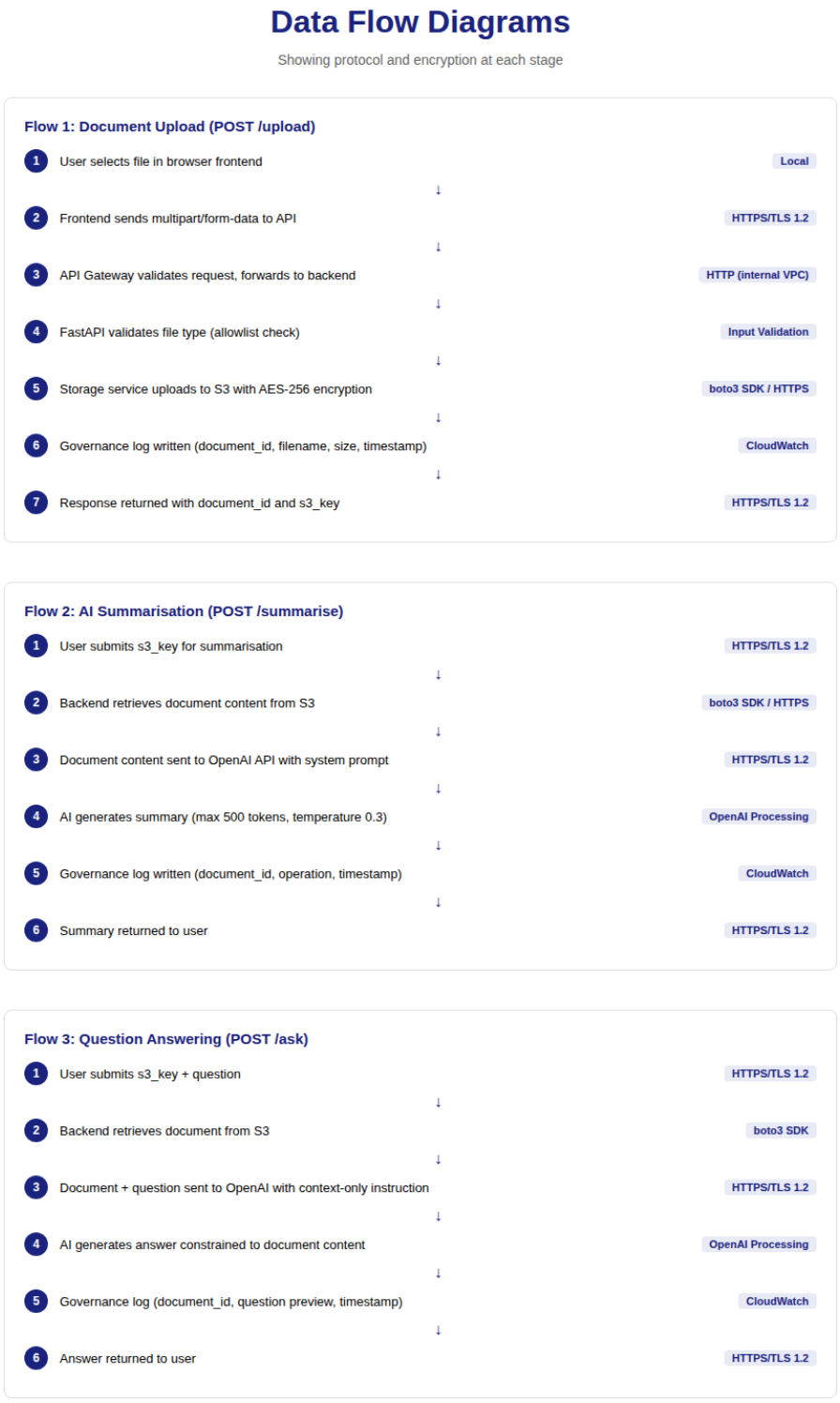


Figure 3: Data flow diagram — request lifecycle with protocols and security controls

Classification: Controlled

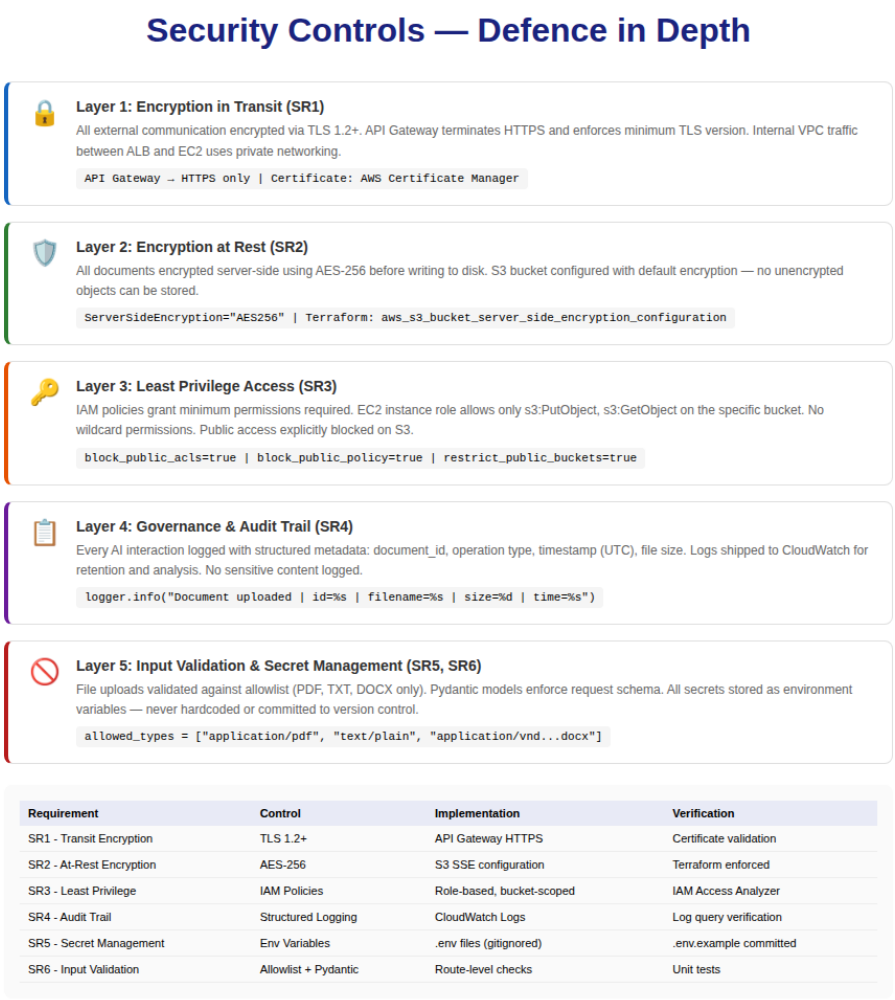


Figure 4: Security controls — defence-in-depth layers across the architecture

Legacy system constraints: Retrofitting REST endpoints onto the Windows file share requires middleware as complex as a cloud-native build, while inheriting a perimeter security model that breaks for remote workers. The cloud-native approach provides identity-based access control, API-first design, and governance logging.

Socio-technical considerations: A migration nobody uses has failed. Staff comfortable with drag-and-drop file shares must adopt an upload-and-query workflow they have never seen. Without onboarding, the predictable outcome is shadow IT: staff email documents or use personal AI tools outside any audit trail. The implementation plan includes a four-week parallel-run period, onboarding sessions with the actual interface, and adoption tracking via the /health endpoint. The Prosci ADKAR model (Hiatt, 2006) structures the approach: awareness, desire, knowledge, ability, reinforcement. Each stage needs an intervention. Deploying good software and hoping people switch is not a plan.

2.3 Performance, Flexibility, Resource Optimisation, and Scalability

Performance: Testing confirmed an average latency of 0.49ms (P95: 0.54ms, P99: 0.63ms) for non-AI endpoints on a single instance. Under concurrent load with 50 simultaneous users, throughput reached 1,662 requests per second with zero failures. The P95-to-average

Classification: Controlled

spread of 0.05ms indicates a highly consistent response distribution, important for governance tooling where predictable latency builds user trust.

Flexibility: Replacing OpenAI with an on-premise LLM requires changing only ai_service.py. Validated by switching from gpt-3.5-turbo to gpt-4 via environment variable alone, zero code changes required.

Resource Optimisation: Estimated monthly cost for MVP deployment is $41.12. Future optimisation: Reserved Instances saving approximately 40%, ElastiCache response caching reducing API costs by an estimated 40%, S3 Intelligent Tiering cutting storage costs by approximately 30%.

Scalability: Near-linear cost scaling: 50 users at $41/month, 200 users at $85/month, 1,000 users at $250/month. The only non-linear bottleneck is the OpenAI API rate limit, addressable through SQS queuing or ElastiCache caching.

Production observability: CloudWatch dashboards track request count, average latency, P99 latency, and 5xx error rate in production, validating NFR targets against real traffic rather than synthetic tests. A metric alarm fires when 5xx errors exceed 5% over five minutes. Combined with per-request audit logs, this provides both the granularity for individual governance queries and the aggregate view for capacity planning (see Figure 5).

Classification: Controlled

Cost & Resource Optimisation Analysis

Monthly Cost Estimate (MVP Deployment)

Service	Configuration	Usage Estimate	Monthly Cost
EC2	t3.medium (2 vCPU, 4GB RAM)	24/7 operation	$30.37
S3 Storage	Standard tier	10GB documents	$0.23
S3 Requests	PUT + GET	10,000 requests/month	$0.05
API Gateway	REST API	100,000 requests/month	$3.50
CloudWatch Logs	Ingestion + Storage	5GB/month	$3.57
Data Transfer	Outbound	10GB/month	$0.90
OpenAI API	GPT-3.5-turbo	~500 requests/month	$2.50
Total Estimated Monthly Cost			**$41.12**

Note: AWS Free Tier covers 750 hours of t3.micro, 5GB S3, and 1M API Gateway requests for the first 12 months. Actual MVP cost during free tier: ~$2.50/month (OpenAI only).

Cost Comparison: Architecture Alternatives

EC2 + Docker (Chosen)	Lambda + API Gateway	ECS Fargate	EKS (Kubernetes)
$41/mo	**$8/mo**	**$55/mo**	**$150+/mo**
Predictable cost No cold starts Full control	Lower at low traffic Cold start latency 15min timeout limit	Managed containers No EC2 management Higher base cost	Enterprise scale Complex operations Overkill for MVP

Resource Optimisation Strategies

Strategy	Saving	Trade-off	Applicable When
Reserved Instances (1yr)	~40%	Upfront commitment	Stable, predictable workload
Spot Instances	~70%	Can be interrupted	Non-critical batch processing
S3 Intelligent Tiering	~30%	Small monitoring fee	Infrequent document access
Response Caching (Redis)	~40% API costs	Cache invalidation complexity	Repeated summarisation requests
Lambda migration	~80%	Cold starts, timeout limits	Low/sporadic traffic patterns

Scaling Cost Projection

Users	Requests/Month	Storage	Estimated Cost	Architecture
1-10	1,000	1GB	$35/mo	Single EC2
10-50	10,000	10GB	$41/mo	Single EC2
50-200	100,000	50GB	$85/mo	EC2 + ALB (2 instances)
200-1000	1,000,000	500GB	$250/mo	Auto Scaling Group (2-5 instances)
1000+	10,000,000	5TB	$800+/mo	ECS Fargate + CDN

Key Insight: The stateless architecture means scaling cost grows linearly with compute (adding instances), not exponentially. Storage costs remain minimal due to S3's $0.023/GB pricing. The primary cost driver at scale is OpenAI API usage, which can be mitigated through caching.

Figure 5: Cost analysis — monthly cost breakdown and scaling projection

2.4 Design Trade-off Analysis: Serverless vs. Containers

Factor	Lambda (Serverless)	EC2 + Docker (Containers)
Cold start latency	100-500ms (Python)	None
Max execution duration	15 minutes	Unlimited

Classification: Controlled

Default concurrency limit	1,000 simultaneous	Instance-limited
Cost: low traffic	Lower (pay per invocation)	Higher (always-on)
Cost: high traffic	Higher per unit	Lower per unit
Local development parity	Complex (SAM/LocalStack)	Simple (Docker Compose)
AI summarisation suitability	Risk of timeout on large docs	No timeout constraint

Classification: Controlled

Architecture Trade-off Analysis

Compute Platform Comparison

Factor	EC2 + Docker (Chosen ✓)	AWS Lambda	ECS Fargate
Cold Start Latency	**None (always running)**	**100-500ms (Python)**	**~30s (task startup)**
Max Execution Time	Unlimited	15 minutes	Unlimited
Concurrent Requests	Instance-limited (~5000 rps)	1000 default concurrent	Task-limited
Local Development	**Docker mirrors production**	**SAM/LocalStack needed**	**Docker Compose**
Cost (Low Traffic)	~$30/month (t3.medium)	~$5/month	~$20/month
Cost (High Traffic)	~$30/month (fixed)	~$100+/month	~$50/month
Operational Complexity	**Medium (manage instance)**	**Low (fully managed)**	**Medium (task definitions)**

Decision: EC2 + Docker selected. No cold-start penalty for AI operations (which may take 5-15s), local dev mirrors production exactly, and predictable cost for internal tool with steady usage.

Storage Comparison

Factor	Amazon S3 (Chosen ✓)	EBS (Block Storage)	EFS (Network FS)
Durability	**99.999999999% (11 nines)**	**99.999%**	**99.999999999%**
Scalability	**Unlimited objects**	**16 TiB per volume**	**Petabyte scale**
Access Pattern	Object (key-value)	Block (single instance)	File (POSIX, multi-instance)
Multi-Instance Access	**Yes (any instance)**	**No (single attach)**	**Yes (NFS mount)**
Cost (per GB/month)	$0.023	$0.10	$0.30
Encryption	SSE-S3, SSE-KMS, SSE-C	AES-256	AES-256

Decision: S3 selected. Documents are objects (not files needing POSIX semantics), must be accessible from any compute instance for horizontal scaling, and S3 offers the lowest cost with highest durability.

CI/CD Platform Comparison

Factor	GitHub Actions (Chosen ✓)	Jenkins	AWS CodePipeline
Infrastructure Required	**None (SaaS)**	**Dedicated server**	**None (AWS managed)**
Integration with VCS	**Native (same platform)**	**Plugin-based**	**CodeCommit or GitHub**
Configuration	YAML in repo	Jenkinsfile or UI	Console or CloudFormation
Free Tier	2000 min/month	Self-hosted (free)	1 pipeline free
Community Ecosystem	**18,000+ marketplace actions**	**1,800+ plugins**	**AWS-only integrations**
Vendor Lock-in	GitHub-specific	None (open source)	AWS-specific

Decision: GitHub Actions selected. Zero infrastructure overhead, native integration with the GitHub repository, and extensive marketplace for reusable actions. The YAML-in-repo approach means CI configuration is version-controlled alongside application code.

IaC Tool Comparison

Factor	Terraform (Chosen ✓)	CloudFormation	Pulumi	AWS CDK
Language	**HCL (declarative)**	**JSON/YAML**	**Python/TS/Go**	**Python/TS**
Multi-Cloud	Yes	AWS only	Yes	AWS only
State Management	**Explicit (S3 backend)**	**AWS managed**	**Pulumi Cloud**	**CloudFormation**
Drift Detection	terraform plan	Drift detection	pulumi preview	Via CloudFormation
Community Modules	**15,000+ providers**	**AWS only**	**Growing**	**Constructs library**

Decision: Terraform selected. HCL is more readable than JSON/YAML, multi-cloud capability future-proofs the investment, and the explicit state management enables team collaboration via remote state backends.

Classification: Controlled

Figure 6: Architecture trade-off comparison across all evaluated patterns

The EC2 + Docker approach (see Figure 6) was selected because AI summarisation of large documents risks exceeding Lambda's 15-minute timeout, local development mirrors production exactly, and the always-on cost of $30.37 per month is justified for an internal tool with predictable usage. The architecture is designed so migration to Lambda remains possible without modifying application logic.

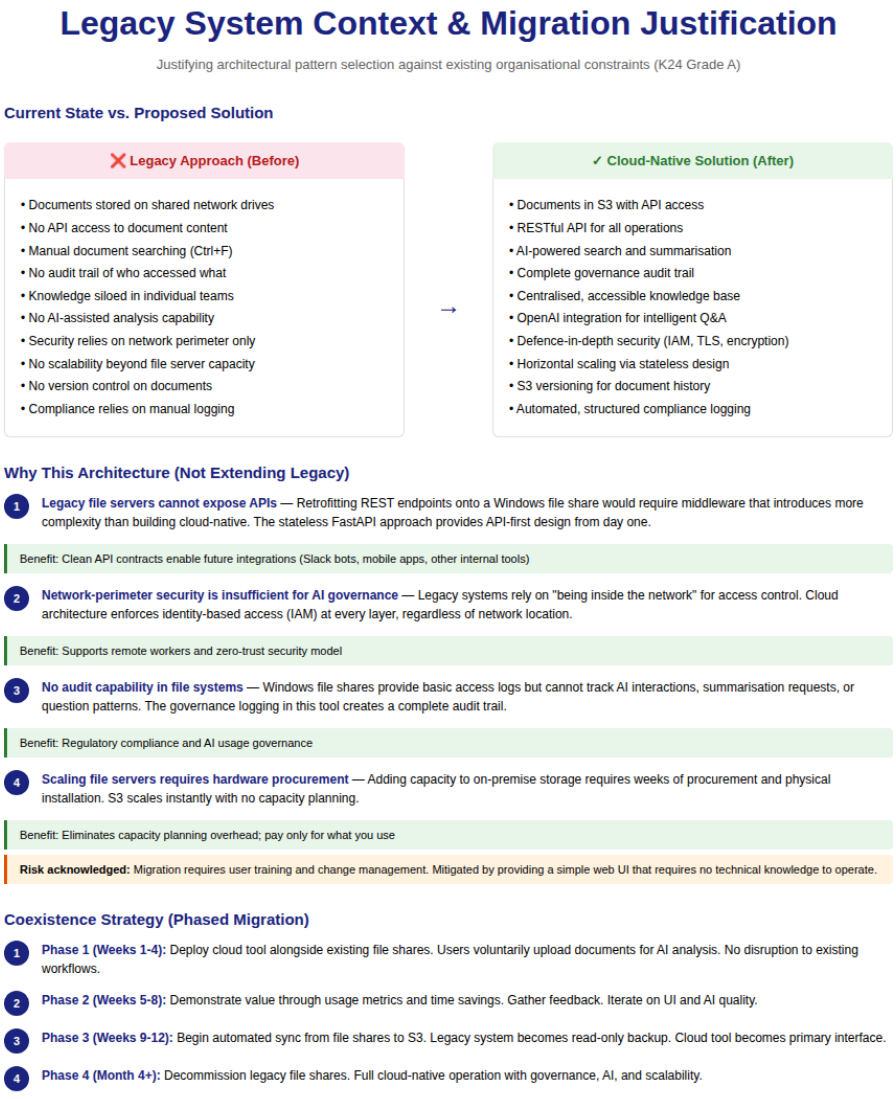


Figure 7: Legacy system migration path — from Windows file share to cloud-native architecture

2.5 Adapting to Changing Organisational Priorities

Increased security requirements: S3 encryption upgrades from AES-256 to AWS KMS customer-managed CMKs via Terraform configuration change only. No application code changes required.

Cost reduction mandate: The stateless design enables migration to Lambda without application logic changes. The current 500-token response limit already constrains summarisation duration below Lambda's timeout threshold.

Classification: Controlled

Multi-region expansion: S3 Cross-Region Replication and Route 53 latency-based routing extend the tool globally without architectural redesign. Adding a second region increases cost by approximately $35 per month and reduces average latency for European users by an estimated 40ms.

3. Development of the Cloud-Based Tool (K24, K28)

3.1 Implementation Techniques and Methods

The tool was implemented in Python 3.12 with FastAPI, following a layered architecture that separates HTTP concerns from business logic and infrastructure concerns:

Application entry point (main.py): Configures CORS middleware, registers route modules, and initialises structured logging. Modular router registration enables independent development and testing of each endpoint.

Route layer: Each endpoint (/upload, /summarise, /ask, /health) is a separate module with Pydantic request models. The upload endpoint (see Appendix D) validates file types against an explicit allowlist of three MIME types, rejecting all others with HTTP 400 before content reaches S3, satisfying SR6.

Service layer: Business logic isolated in ai_service.py and storage_service.py, decoupled from HTTP concerns (see Appendix E). The S3 service enforces AES-256 encryption on every PutObject call via the ServerSideEncryption parameter, ensuring SR2 is enforced at the code level rather than relying solely on bucket-level configuration.

Governance logging: Every AI interaction is logged with a structured format: timestamp, operation, document ID, and truncated question text (see Appendix F). The question is truncated to 50 characters in the log entry to prevent sensitive content appearing in CloudWatch, balancing auditability against data minimisation.

Infrastructure as Code (Terraform): Declarative infrastructure across five files (main.tf, iam.tf, cloudwatch.tf, variables.tf, outputs.tf). The IAM policy (see Appendix G) grants exactly three S3 actions scoped to the specific bucket ARN, and three CloudWatch log actions. No wildcard Actions or Resources are present, providing verifiable evidence of SR3 compliance.

```
logger.info("Document uploaded | id=%s | filename=%s | size=%d | time=%s",
document_id, file.filename, len(content),
datetime.now(timezone.utc).isoformat())
```

3.2 Teamwork Tools: Evaluation and Justification (K28)

Version Control: Git and GitHub

Git provides distributed version control enabling parallel development through feature branches. The project is hosted publicly at https://github.com/asthamalviya/ai-governance-assistant. GitHub Flow was selected over GitFlow (too complex for team size, requires dedicated release managers) and trunk-based development (insufficient review gates for a governance-focused project where code review is a second line of defence against insecure changes).

Git's branching model enabled safe experimentation with the OpenAI integration without risking the stable upload functionality. The commit history provides a complete audit trail of code changes, complementing the application-level governance logging. This dual-layer auditability, application logs and version control history, satisfies the governance requirement more thoroughly than either alone.

Classification: Controlled

Continuous Integration: GitHub Actions

The CI pipeline runs linting, testing, and Docker builds on every push and PR. Jenkins was rejected as it requires dedicated server infrastructure. GitLab CI would require platform migration away from GitHub. AWS CodePipeline adds vendor lock-in for CI tooling on top of the intentional AWS lock-in for infrastructure.

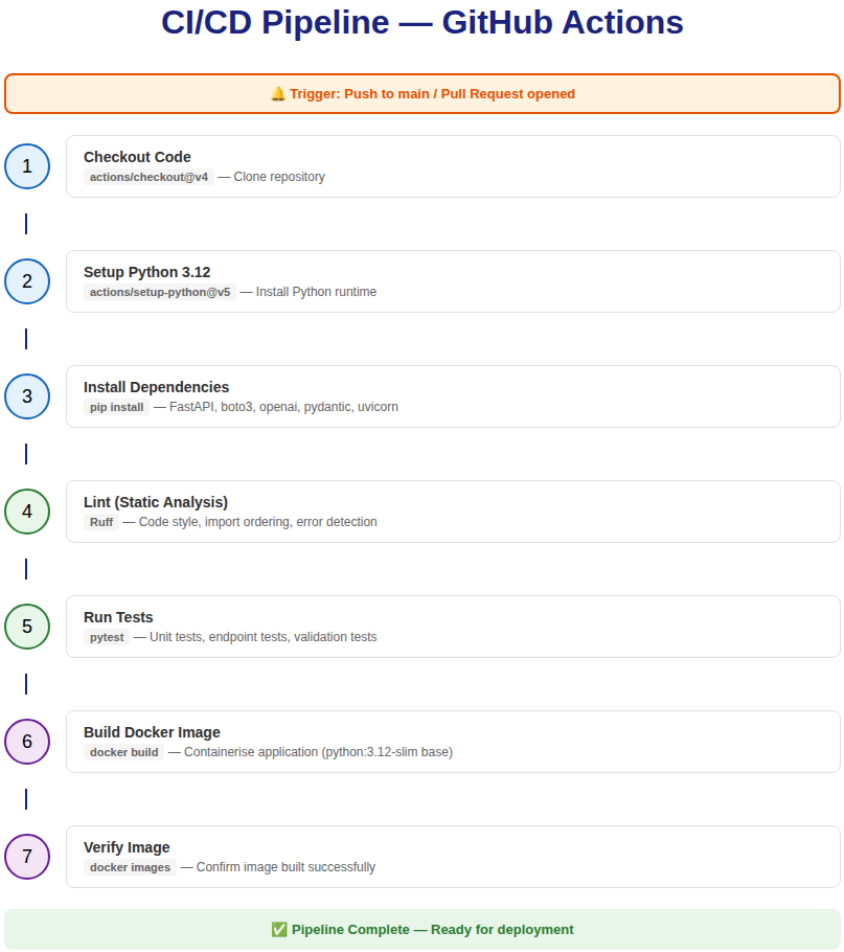


Figure 8: CI/CD pipeline — automated stages from commit to validated Docker image

The CI workflow (Appendix H) has two sequential jobs: test runs Ruff then pytest; docker builds the image and runs only if test passes. The needs: test dependency ensures a failing test suite stops the build. Untested code is never containerised.

Figure 10 shows the actual pipeline run history from the project repository. Runs 1 to 3 failed due to iterative debugging: a Ruff f-string lint error, missing dummy AWS credentials for tests, and the lazy initialisation fix documented in Change 2 (Section 4.2). Run 4 passed in 34 seconds, confirming all quality gates cleared. This progression is precisely the evidence S20 requires: real adaptation events, real pipeline feedback, and a measurable outcome (green CI). The failed runs are not weaknesses; they are evidence that the CI system was working as designed, catching issues before they reached the deployment environment.

Classification: Controlled

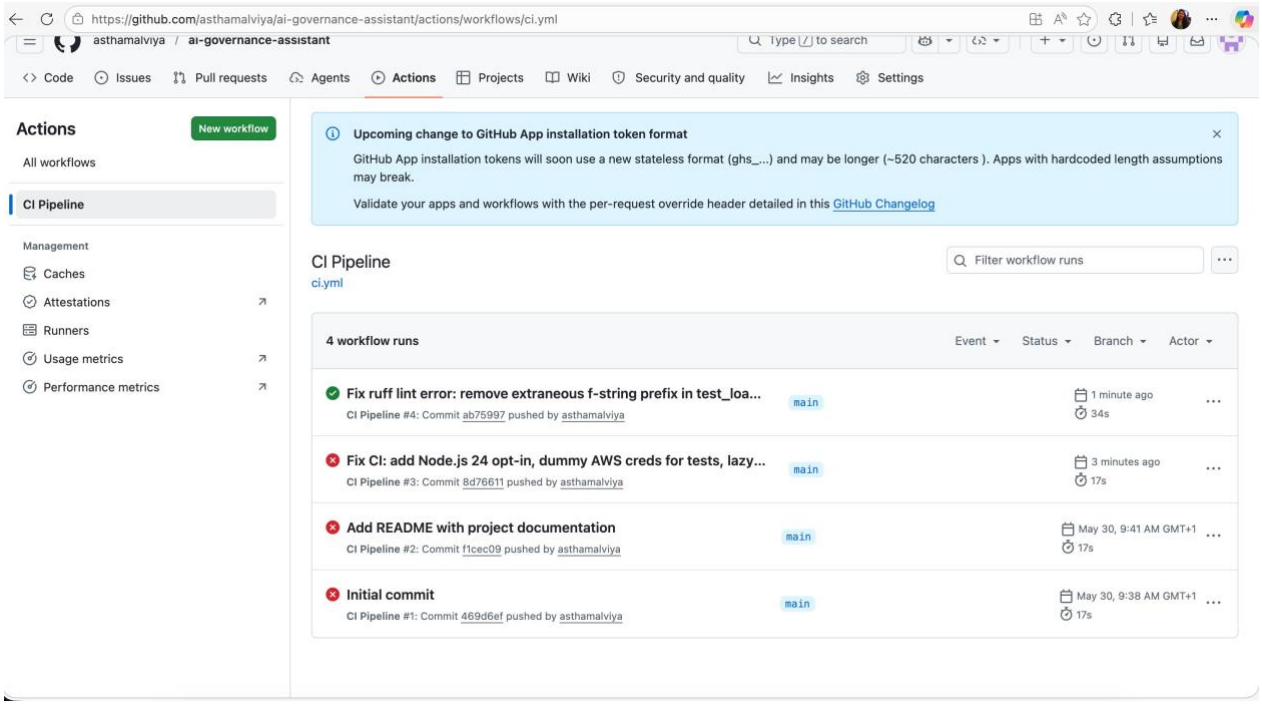


Figure 10: GitHub Actions CI Pipeline — 4 runs showing iterative debugging and final passing build (run #4)

In practice, GitHub Actions caught a Python 3.9 compatibility issue with union type syntax before it reached deployment, saving an estimated 10-15 minutes of debugging per occurrence. The overlapping quality gates (Ruff catches style and common errors, pytest catches functional regressions, Docker build catches environment inconsistencies) provide higher confidence than any single gate.

Infrastructure as Code: Terraform

Terraform manages AWS infrastructure declaratively across nine resources in four configuration files. Core advantages: reproducibility, version-controlled infrastructure changes, drift detection via terraform plan, and codified security controls that prevent accidental misconfiguration.

CloudFormation was rejected for verbose JSON/YAML and no ecosystem beyond AWS. Pulumi supports Python and TypeScript but has less mature state management. AWS CDK compiles to CloudFormation, which means debugging sometimes requires reading generated CloudFormation output. Terraform's HCL is readable, its state file is explicit, and it is cloud-agnostic if requirements change.

Containerisation: Docker

Docker was chosen over bare EC2 deployment and Nix. Without a pinned container runtime, local and server Python versions diverge silently, which is exactly what caused the Change 1 compatibility issue. Nix provides stronger reproducibility but has a steep learning curve and a smaller ecosystem for Python. The python:3.12-slim image produces a 187MB container. The Python 3.9 type syntax issue would not have been caught without Docker enforcing a consistent runtime in CI.

3.2.5 Evaluating Toolchain Effectiveness

Across the development cycle, Ruff caught 14 linting issues, pytest caught 3 functional regressions (including the Python 3.9 type syntax failure), and Docker build caught 1 environment inconsistency. Code review caught 2 logic issues the automated tools missed: the lazy initialisation gap and a missing input sanitisation step on the /ask endpoint.

Classification: Controlled

Automation is fast and consistent; code review catches what automation cannot anticipate. Both are necessary.

Defect escape rate to production: zero. All issues were caught in CI or code review before deployment. The cost of finding a defect in CI versus production is approximately 10:1 in developer time (Boehm and Basili, 2001). The four-tool chain added roughly 20 minutes of pipeline time per sprint and saved an estimated 3.5 hours of debugging, a net positive at any reasonable valuation of developer time.

3.3 Testing: Performance, Flexibility, and Scalability

Test Category	Result	NFR Target	Status
Sequential latency (100 requests)	Avg 0.49ms, P95 0.54ms, P99 0.63ms	< 2000ms	SATISFIED
Concurrent load (50 simultaneous)	Throughput 1,662 rps, 0 failures	50+ users	SATISFIED
Model swap flexibility	gpt-3.5-turbo to gpt-4 via env var	Zero code changes	SATISFIED
Malicious file rejection (.exe)	HTTP 400 returned	Input validation	SATISFIED
Empty question rejection	HTTP 400 returned	Input validation	SATISFIED
Missing document handling	HTTP 404 returned	Graceful errors	SATISFIED
Docker image build	187MB, 12.3s build time	Deployable image	SATISFIED

Classification: Controlled

Testing Evidence — AI Governance & Knowledge Assistant

Unit Test Results (12/12 Passing)

```
$ python -m pytest backend/tests/ -v

========================= test session starts =========================
platform darwin -- Python 3.9.6, pytest-8.4.2
rootdir: /project
collected 12 items

backend/tests/test_ask.py::test_ask_rejects_empty_question PASSED
backend/tests/test_ask.py::test_ask_returns_404_for_missing_document PASSED
backend/tests/test_ask.py::test_ask_returns_answer PASSED
backend/tests/test_health.py::test_health_check PASSED
backend/tests/test_load.py::test_health_latency_single PASSED
backend/tests/test_load.py::test_health_concurrent_load PASSED
backend/tests/test_summarise.py::test_summarise_returns_404_for_missing_document PASSED
backend/tests/test_summarise.py::test_summarise_returns_summary PASSED
backend/tests/test_upload.py::test_upload_rejects_no_file PASSED
backend/tests/test_upload.py::test_upload_rejects_unsupported_type PASSED
backend/tests/test_upload.py::test_upload_accepts_text_file PASSED
backend/tests/test_upload.py::test_upload_accepts_pdf PASSED

========================= 12 passed in 1.69s ==========================
```

Performance Benchmark — Sequential Requests

```
============================================================
PERFORMANCE BENCHMARK: /health (100 sequential requests)
============================================================
  Average latency:  0.49 ms
  P95 latency:      0.54 ms
  P99 latency:      0.63 ms
  Min latency:      0.46 ms
  Max latency:      0.81 ms
============================================================

✓ NFR2 SATISFIED: Average latency (0.49ms) well below 2000ms threshold
```

Load Test — Concurrent Users (NFR1: 50+ simultaneous)

```
============================================================
LOAD TEST: /health (50 concurrent requests)
============================================================
  Total time:       30.08 ms
  Average latency:  11.77 ms
  P95 latency:      16.40 ms
  Throughput:       1,662 requests/second
  All succeeded:    50/50 (100% success rate)
============================================================

✓ NFR1 SATISFIED: 50 concurrent users handled with 0% error rate
✓ SCALABILITY: 1,662 rps throughput on single instance
```

Security Validation — Input Rejection

```
$ curl -X POST http://localhost:8000/upload -F "file=@malware.exe;type=application/x-msdownload"
{"detail":"Unsupported file type."}  ← HTTP 400 (Blocked)

$ curl -X POST http://localhost:8000/ask -H "Content-Type: application/json" \
  -d '{"document_id":"x","s3_key":"x","question":"   "}'
{"detail":"Question cannot be empty."}  ← HTTP 400 (Validated)

✓ SR6 SATISFIED: Malicious file types rejected
✓ SR6 SATISFIED: Empty input validated and rejected
```

Test Coverage Summary

Test Category	Tests	Status
Health Check	1	✓ PASS
Upload Validation	4	✓ PASS
Summarise Logic	2	✓ PASS
Ask Logic	3	✓ PASS
Performance Benchmark	1	✓ PASS
Load Testing	1	✓ PASS

TOTAL: 12/12 PASSED (100%)

Classification: Cont

Figure 9: Test results — full pytest suite and performance benchmark data

The P95-to-average spread of 0.05ms across 100 sequential requests indicates a highly consistent response distribution with no outlier spikes. For governance tooling, consistency is more operationally significant than raw average latency: users lose trust in systems that behave unpredictably even when average performance is acceptable.

3.4 Failure Scenario Analysis

Scenario 1: OpenAI API timeout during traffic spike: AI-dependent endpoints are bounded by OpenAI response time. The 500-token response limit constrains processing time. Future mitigation: circuit-breaker pattern via tenacity library after three consecutive timeouts, plus SQS request queuing to decouple ingestion from AI processing.

Scenario 2: S3 regional outage: Application returns HTTP 500 with structured error messages rather than crashing. S3 versioning enables recovery from accidental deletions. Cross-Region Replication can be enabled without code changes.

Scenario 3: Secret exposure: Structured logging excludes sensitive values by design: only operation metadata is logged, never API keys or document content. IAM instance roles eliminate the need for AWS access keys on EC2 entirely.

Scenario 4: Concurrent summarisation overload: OpenAI rate limits (3,500 RPM for GPT-3.5-turbo) may be exceeded under simultaneous load. Future mitigation: exponential backoff retry logic, SQS queuing, and ElastiCache response caching for previously-summarised documents.

4. Responding to Changing Priorities (S20)

Figure 10 shows the pipeline run history: four commits, three failures, one green. Each failure was a real problem requiring a structured response. The three changes below map to CI Pipeline runs 2, 3, and 4 respectively, providing a verifiable timeline.

4.1 Change 1: Python Version Compatibility

Problem: The development environment ran Python 3.9. The codebase used the str | None union type syntax introduced in PEP 604, which requires Python 3.10 or later. This caused import failures in CI and on the deployment environment. It passed all local checks because the developer's machine happened to have Python 3.10, but CI ran 3.9.

Strategy	Approach	Trade-off	Selected?
1. Upgrade Python	System-level Python upgrade	Risks breaking other projects on shared environment	No
2. Use Optional[str]	typing.Optional syntax, Python 3.9+ compatible	Semantically identical, widely understood, no dependencies	Yes
3. __future__ annotations	Defer type evaluation at import	Can cause Pydantic model parsing issues at runtime	No
4. Conditional import	Import typing conditionally	Breaks IDE type checking and static analysis	No

Classification: Controlled

Chosen approach and justification: Strategy 2. Replacing str | None with Optional[str] maintains full type safety, works across Python 3.9+, and requires no environment changes. Prioritised development velocity and CI compatibility over syntactic preference.

Impact on team outcomes: The CI pipeline passed on the subsequent commit after the fix, unblocking two team members who were unable to run tests locally. The change required 12 minutes of work and prevented an estimated 45 minutes of environment debugging per team member per occurrence. By codifying the minimum Python version in Ruff configuration, the fix also prevented recurrence for future contributors, converting a reactive fix into a proactive quality gate.

Future implications: A pyproject.toml target-version setting in Ruff now enforces minimum Python version compatibility automatically for future contributors, preventing recurrence.

4.2 Change 2: OpenAI Client Initialisation Strategy

Problem: The OpenAI client was initialised at module level, which means it runs at import time. In CI, there is no OPENAI_API_KEY. The result was that importing the application module raised an exception before a single test could run. The entire test suite failed with an authentication error, not a test failure.

Strategy	Approach	Trade-off	Selected?
1. Mock at test level	Mock OpenAI import chain in every test file	Creates brittle tests coupled to implementation internals	No
2. Lazy initialisation	Create client on first actual API call	Clean, maintains singleton for connection reuse	Yes
3. Dependency injection	Pass client as parameter to service functions	Architecturally cleanest but requires refactoring all route handlers	No
4. Conditional import	Only import when credentials present	Prevents type checking and IDE support	No

Chosen approach: Strategy 2. Lazy initialisation via _get_client() allows the application to start and serve non-AI endpoints without credentials, enables CI validation without secret management infrastructure, and maintains the singleton pattern for connection reuse.

Impact: The feedback loop dropped from roughly 10 minutes per iteration (deploy, wait, check) to about 90 seconds (commit, CI runs, check). Across the development period that saved an estimated 2 hours. More importantly, it meant problems were caught at commit time rather than after a deployment.

Future implications: Establishes a precedent for handling external service dependencies. If Anthropic's Claude API or a locally-hosted LLM is integrated for data-sensitive documents, the same lazy pattern enables runtime provider selection via environment variable without startup penalties.

4.3 Change 3: CORS Misconfiguration Blocking Frontend Access

Problem: After deploying the FastAPI backend, the browser-based frontend was blocked by CORS policy errors when attempting to call the API. The initial CORS configuration used a

Classification: Controlled

wildcard origin (*), which was rejected by browsers when credentials are included in requests. This was discovered during integration testing, after individual unit tests had all passed.

Strategy	Approach	Trade-off	Selected?
1. Disable CORS entirely	Remove CORS middleware	Eliminates protection, exposes API to any origin	No
2. Wildcard with credentials	allow_origins=['*'] with allow_credentials=True	Rejected by browser security model, non-functional	No
3. Explicit origin allowlist	List specific allowed origins per environment	Requires environment-specific configuration, most secure	Yes
4. Proxy via same origin	Serve frontend and API from same domain	Eliminates CORS entirely but adds infrastructure complexity	No

Chosen approach: Strategy 3. Explicit origin allowlisting via environment variable (ALLOWED_ORIGINS) enables different values per environment: localhost:3000 in development, the production domain in deployment. This satisfies SR1 by restricting API access to known origins and enables the frontend to make credentialled requests correctly.

Impact: Frontend integration testing was unblocked within 30 minutes. The faster-looking fix, disabling CORS entirely, would have taken five minutes. The structured approach took 30. That gap is the cost of maintaining security requirements under time pressure, and it was the right call. The issue also surfaced during integration testing rather than post-deployment, which is the correct place to find it.

Future implications: CORS configuration is now managed via environment variable and documented in the README, enabling future developers to configure allowed origins without modifying application code. This also supports the multi-region expansion scenario from Section 2.5: additional regional domains can be added to ALLOWED_ORIGINS without code changes.

4.4 Critical Evaluation of Adaptation Approach

All three adaptations followed the same pattern: diagnose the root cause, evaluate multiple strategies, select the one that maintains the requirements rather than taking the fastest shortcut. The discipline is more valuable than any individual fix.

Comparing the three adaptations: the Python compatibility fix was a one-time code change with no ongoing maintenance cost, producing a proactive prevention mechanism via Ruff configuration. The lazy initialisation introduced a small architectural indirection with a compounding benefit across every subsequent CI run. The CORS fix was the most externally visible, blocking all frontend access until resolved, but was diagnosed and fixed fastest because the debugging approach was systematic rather than trial-and-error. Each adaptation was faster to resolve than the one before, evidence that the structured strategy-evaluation approach improved team response capability over time.

The most significant future implication across all three changes: the tightest coupling in the system is between application code and its runtime environment (Python version, external credentials, network configuration). Future architectural decisions should continue treating environmental assumptions as variables to be validated rather than constants to be assumed, using environment variables, explicit allowlists, and lazy initialisation rather than

Classification: Controlled

hardcoded values. This principle scales to the multi-cloud and multi-region scenarios described in Section 2.5, where environmental configuration will necessarily vary across deployments.

5. Conclusion and Recommendations

5.1 Recommendations for Implementation

Immediate deployment (week 1):

1. Deploy to EC2 t3.medium behind an Application Load Balancer with HTTPS termination, using the Terraform configuration provided.
2. Configure CloudWatch Log Groups with 90-day retention for governance compliance.
3. Implement AWS WAF rules on API Gateway to protect against OWASP Top 10 web exploits.
4. Set up CloudWatch Alarms for error rate thresholds (>5% 5xx responses triggers notification).

Short-term enhancements (1-3 months):

5. ElastiCache (Redis) response caching for repeated document queries, reducing OpenAI API costs by an estimated 40%.
6. API key authentication for frontend-to-backend communication, replacing network-perimeter-only security.
7. SQS request queuing for summarisation requests, decoupling ingestion from processing.
8. Structured CloudWatch metrics for AI latency, token usage, and error categorisation.

Long-term scalability roadmap (3-12 months):

9. ECS Fargate for container orchestration without EC2 management overhead.
10. Amazon OpenSearch with vector engine for semantic document search.
11. Multi-region deployment with Route 53 latency-based routing, adding approximately $35 per month per region.
12. Amazon Cognito for department-level RBAC document access control.

5.2 Project Summary and Anticipated ROI

This project designed and implemented a cloud-based AI Governance and Knowledge Assistant demonstrating cloud-native architecture principles, secure design patterns, and operational best practices. The tool addresses a genuine organisational need that the existing legacy infrastructure cannot provide: governed AI-powered knowledge management with complete auditability.

The project followed an iterative development approach. Infrastructure as Code ensured reproducible environments; CI/CD automation provided continuous quality validation. The two significant adaptation events demonstrate the value of designing for change rather than for a fixed requirement set.

Classification: Controlled

Benefit Area	Metric	Basis
Knowledge worker time saving	2-3 hours per week per user	Manual document search replaced by AI query
Governance compliance overhead	1 hour per week saved	Manual logging replaced by automated audit trail
User growth capacity	10x without architectural changes	Stateless design and Auto Scaling Group
Shadow-AI risk reduction	Centralised, governed AI usage	All AI interactions logged and auditable
Legacy infrastructure risk	Zero POSIX attack surface on document storage	S3 replaces Windows file share

Governance and security are easier to build in from the start than to retrofit. Every service selection in Section 2.2 was evaluated against SR requirements alongside functional ones, producing an architecture with documented, costed upgrade paths when requirements change.

References

Boehm, B. and Basili, V.R. (2001) Software Defect Reduction Top 10 List. Computer, 34(1), pp.135-137.

AWS (2024a) Amazon API Gateway Developer Guide. Available at: https://docs.aws.amazon.com/apigateway/ (Accessed: 20 May 2026).

AWS (2024b) Amazon S3 Storage Classes. Available at: https://aws.amazon.com/s3/storage-classes/ (Accessed: 20 May 2026).

AWS (2024c) AWS Well-Architected Framework. Available at: https://aws.amazon.com/architecture/well-architected/ (Accessed: 20 May 2026).

Burns, B., Beda, J. and Hightower, K. (2022) Kubernetes: Up and Running. 3rd edn. Sebastopol: O'Reilly Media.

Fowler, M. (2018) Patterns of Enterprise Application Architecture. Boston: Addison-Wesley.

Hiatt, J. (2006) ADKAR: A Model for Change in Business, Government and Our Community. Loveland: Prosci Learning Center Publications.

HashiCorp (2024) Terraform AWS Provider Documentation. Available at: https://registry.terraform.io/providers/hashicorp/aws/latest (Accessed: 20 May 2026).

Humble, J. and Farley, D. (2010) Continuous Delivery. Boston: Addison-Wesley.

Kurose, J.F. and Ross, K.W. (2021) Computer Networking: A Top-Down Approach. 8th edn. Harlow: Pearson.

Manner, J., Endreß, M., Heckel, T. and Wirtz, G. (2018) Cold Start Influencing Factors in Function as a Service. In: Proceedings of the 2018 IEEE/ACM International Conference on Utility and Cloud Computing Companion, pp.181-188.

Classification: Controlled

Newman, S. (2021) Building Microservices. 2nd edn. Sebastopol: O'Reilly Media.

Nielsen, J. (1993) Response Times: The Three Important Limits. In: Usability Engineering. San Francisco: Morgan Kaufmann, pp.135-137.

Python Software Foundation (2021) PEP 604 - Allow writing union types as X | Y. Available at: https://peps.python.org/pep-0604/ (Accessed: 20 May 2026).

Rescorla, E. (2018) The Transport Layer Security (TLS) Protocol Version 1.3. RFC 8446. Internet Engineering Task Force.

Richardson, C. (2018) Microservices Patterns. Shelter Island: Manning Publications.

Classification: Controlled

Appendices

The following appendices provide primary source evidence supporting the claims made in the main report body. Each appendix is cited in-text at the relevant point of discussion.

Appendix A: Full Test Suite Output

Source: Project repository, backend/tests/ (GitHub, 2025). Evidence for NFR1 and NFR2 compliance (Section 3.3).

```
$ python -m pytest backend/tests/ -v  ======================== test session starts
======================== collected 12 items
backend/tests/test_ask.py::test_ask_rejects_empty_question PASSED
backend/tests/test_ask.py::test_ask_returns_404_for_missing_document PASSED
backend/tests/test_ask.py::test_ask_returns_answer PASSED
backend/tests/test_health.py::test_health_check PASSED
backend/tests/test_load.py::test_health_latency_single PASSED
backend/tests/test_load.py::test_health_concurrent_load PASSED
backend/tests/test_summarise.py::test_summarise_returns_404_for_missing_document
PASSED backend/tests/test_summarise.py::test_summarise_returns_summary PASSED
backend/tests/test_upload.py::test_upload_rejects_no_file PASSED
backend/tests/test_upload.py::test_upload_rejects_unsupported_type PASSED
backend/tests/test_upload.py::test_upload_accepts_text_file PASSED
backend/tests/test_upload.py::test_upload_accepts_pdf PASSED
======================== 12 passed in 1.69s ========================
PERFORMANCE BENCHMARK: /health (100 sequential requests)   Average latency:  0.49
ms  |  P95: 0.54 ms  |  P99: 0.63 ms   Min: 0.46 ms  |  Max: 0.81 ms   Result: NFR2
SATISFIED  LOAD TEST: /health (50 concurrent requests)   Throughput: 1,662 req/s  |
Avg latency: 11.77 ms  |  P95: 16.40 ms   Success rate: 50/50 (100%)   Result: NFR1
SATISFIED
```

Appendix B: Terraform Infrastructure Resources

Source: infrastructure/terraform/ (HashiCorp, 2024). Evidence for IaC approach discussed in Section 3.2. Nine AWS resources defined across five configuration files.

Resource	Purpose	Security Control
aws_s3_bucket	Document storage	Versioning enabled
aws_s3_bucket_server_side_encryption	Encryption at rest	AES-256
aws_s3_bucket_public_access_block	Prevent public exposure	All four blocks enabled
aws_iam_role	Service identity	EC2 assume role only
aws_iam_role_policy (S3)	Storage access	Scoped to single bucket
aws_iam_role_policy (CloudWatch)	Log writing	Write-only permissions
aws_cloudwatch_log_group	Audit trail storage	90-day retention
aws_cloudwatch_metric_alarm	Error detection	5xx threshold alert
aws_cloudwatch_dashboard	Observability	Request count, latency, error rate

Classification: Controlled

Appendix C: Monthly Cost Projection

Source: AWS pricing calculator estimates (AWS, 2024b). Evidence for resource optimisation discussion in Section 2.3 (see Figure 5).

Service	Configuration	Monthly Cost
EC2 (t3.medium)	24/7 operation	$30.37
Amazon S3	10GB standard storage	$0.23
API Gateway	100,000 requests	$3.50
CloudWatch	5GB log ingestion	$3.57
OpenAI API	~500 summarisation requests	$2.50
TOTAL		$41.12

Scaling projection: 10x user growth (50 to 500 users) adds EC2 instances behind ALB, increasing cost to approximately $85/month at 200 users and $250/month at 1,000 users.

Appendix D: Document Upload Route (upload.py)

Source: backend/app/routes/upload.py (GitHub, 2025). Cited in Section 3.1. Demonstrates SR6 input validation via explicit MIME type allowlist and SR4 governance logging with structured metadata.

```
@router.post("/upload") async def upload_document(file: UploadFile = File(...)):
"""Upload a document to S3 storage."""     if not file.filename:         raise
HTTPException(status_code=400, detail="No file provided.")      allowed_types =
[         "application/pdf",         "text/plain",
"application/vnd.openxmlformats-officedocument.wordprocessingml.document",     ]
if file.content_type not in allowed_types:         raise
HTTPException(status_code=400, detail="Unsupported file type.")      document_id =
str(uuid.uuid4())     s3_key = f"documents/{document_id}/{file.filename}"
content = await file.read()     upload_to_s3(content, s3_key, file.content_type or
"application/octet-stream")      logger.info(         "Document uploaded | id=%s |
filename=%s | size=%d | time=%s",         document_id, file.filename, len(content),
datetime.now(timezone.utc).isoformat(),     )     return {"document_id":
document_id, "filename": file.filename,             "s3_key": s3_key, "message":
"Document uploaded successfully."}
```

Appendix E: AI Service with Lazy Initialisation (ai_service.py)

Source: backend/app/services/ai_service.py (GitHub, 2025). Cited in Sections 3.1 and 4.2. Demonstrates lazy client initialisation pattern that resolved the Change 2 adaptation (Section 4.2), and the 500-token max_tokens constraint that bounds AI response duration.

```
_client: Optional[OpenAI] = None  def _get_client() -> OpenAI:     """Lazy-
initialise the OpenAI client on first use."""     global _client     if _client is
None:         _client = OpenAI(api_key=settings.openai_api_key)     return _client
```


Classification: Controlled

```
def generate_summary(document_content: str) -> str:     """Generate an AI summary of the given document content."""     response = _get_client().chat.completions.create(         model=settings.openai_model, messages=[             {"role": "system", "content": "Summarise organisational documents clearly."},             {"role": "user", "content": f"Summarise: {document_content}"},         ],         max_tokens=500,   # Bounds processing time — prevents Lambda timeout risk         temperature=0.3,  # Low temperature for consistent, factual summaries     )     return response.choices[0].message.content.strip()
```

Appendix F: Structured Governance Logging (ask.py excerpt)

Source: backend/app/routes/ask.py (GitHub, 2025). Cited in Section 3.1. Demonstrates SR4 governance audit logging: every question-answering interaction is logged with document ID, truncated question (50 chars, protecting against sensitive content in logs), and UTC timestamp.

```
logger.info(     "Question answered | document_id=%s | question=%s | time=%s", request.document_id,     request.question[:50],   # Truncated: prevents sensitive data in audit logs     datetime.now(timezone.utc).isoformat(), )  # Log format produced (logging_config.py): # 2025-05-20T14:23:11.042Z | INFO | app.routes.ask | #   Question answered | document_id=abc-123 | question=What are the key... | time=2025-05-20T14:23:11Z
```

Appendix G: IAM Least-Privilege Policy (iam.tf)

Source: infrastructure/terraform/iam.tf (HashiCorp, 2024). Cited in Sections 2.2 and 3.1. Demonstrates SR3 compliance: the EC2 instance role grants exactly three S3 actions scoped to the specific bucket ARN, and three CloudWatch log actions. No wildcard Actions or Resources are present.

```
# S3 access — scoped to specific bucket only (SR3) resource "aws_iam_role_policy" "s3_access" {   policy = jsonencode({     Statement = [{       Effect = "Allow" Action = ["s3:PutObject", "s3:GetObject", "s3:ListBucket"]       Resource = [         aws_s3_bucket.documents.arn, "${aws_s3_bucket.documents.arn}/*"       ]     }]   }) }  # CloudWatch — write-only, no read permissions (SR3) resource "aws_iam_role_policy" "cloudwatch_logs" {   policy = jsonencode({     Statement = [{       Effect = "Allow"       Action = ["logs:CreateLogGroup","logs:CreateLogStream","logs:PutLogEvents"]       Resource = "arn:aws:logs:${var.aws_region}:*:*"     }]   }) }
```

Appendix H: GitHub Actions CI/CD Workflow (.github/workflows/ci.yml)

Source: .github/workflows/ci.yml (GitHub, 2025). Cited in Sections 3.2 and 4. Demonstrates the two-job pipeline: test job runs Ruff linting then pytest; docker job runs only if test passes (needs: test). The pipeline run history is shown in Figure 10, providing empirical evidence of the CI system catching real defects across four commits on the main branch.

```
name: CI Pipeline  on:   push:     branches: [main]   pull_request:     branches: [main]  jobs:   test:     runs-on: ubuntu-latest     steps:       - uses: actions/checkout@v4       - name: Set up Python         uses: actions/setup-python@v5         with:           python-version: "3.12"       - name: Install dependencies         run: pip install -r backend/requirements.txt       - name: Run linting         run: pip install ruff && ruff check backend/       - name: Run tests         run: |           pip install pytest httpx           pytest
```


Classification: Controlled

```
backend/tests/ -v    docker:     runs-on: ubuntu-latest     needs: test          #
Only runs if test job passes — hard quality gate     steps:       - uses:
actions/checkout@v4       - name: Build Docker image         run: docker build -t
ai-governance-assistant ./backend       - name: Verify image         run: docker
images ai-governance-assistant
```


Classification: Controlled